

Katholieke
Universiteit
Leuven

**Department of
Computer Science**

WARPS, THREAD DIVERGENCE AND STRIDING

Lab 3

Matt Delalieux (r0890338)
Radi Miah (r1012080)

Academic year 2025–2026

Contents

1 Part 1 – Warps and Image Inversion **2**
1.1 Implementation . . . 2
1.1.1 The Inversion Kernel . . . 2
1.1.2 Striding Correctness . . . 2
1.1.3 Timing Methodology . . . 3
1.2 Results and Analysis . . . 3
1.3 Warp Efficiency Considerations . . . 3

2 Part 2 – Thread Divergence **4**
2.1 Implementation . . . 4
2.1.1 Divergent Kernel (Interleaved Layout) . . . 4
2.1.2 Non-Divergent Kernel (Planar Layout) . . . 4
2.2 Results . . . 5
2.3 Analysis . . . 6
2.3.1 Overall Shape . . . 6
2.3.2 Divergent vs. Non-Divergent Gap . . . 7
2.3.3 Effect of Threads per Block . . . 7
2.3.4 Correctness Verification . . . 7
2.4 Conclusions . . . 8

1 Part 1 – Warps and Image Inversion

1.1 Implementation

1.1.1 The Inversion Kernel

The kernel uses a *grid-stride loop* so that any number of launched threads correctly covers all image elements:

```
__global__ void invertKernel(uint8_t* img, int total_elements)
{
    int abs_tid   = blockIdx.x * blockDim.x + threadIdx.x;
    int grid_size = gridDim.x  * blockDim.x;

    for (int idx = abs_tid; idx < total_elements; idx += grid_size) {
        img[idx] = 255 - img[idx];
    }
}
```

Listing 1: Image inversion kernel with grid-stride loop

Each thread starts at its absolute index `abs_tid` and jumps forward by the total grid size on every iteration. This guarantees that every element is processed exactly once, even when far fewer threads than elements are launched.

1.1.2 Striding Correctness

Correctness was verified by running the kernel with a thread-to-element ratio of 1:16 and comparing the output pixel-by-pixel against a fully-threaded run. The output image `output_image.ppm` was visually inspected and compared with an online tool, demonstrating that striding works correctly even at very low thread counts.



(a) Original input image (`anna.ppm`).



(b) Inverted output from Part 1 kernel.

Figure 1: Visual verification of the image inversion kernel from Part 1.

1.1.3 Timing Methodology

Each configuration was timed using CUDA events over 10 independent runs. The experiment used 32 threads per block (exactly one warp per block), sweeping the total thread count from 1 up to the full element count (3,686,400 bytes for the 960×1280 RGB image).

1.2 Results and Analysis

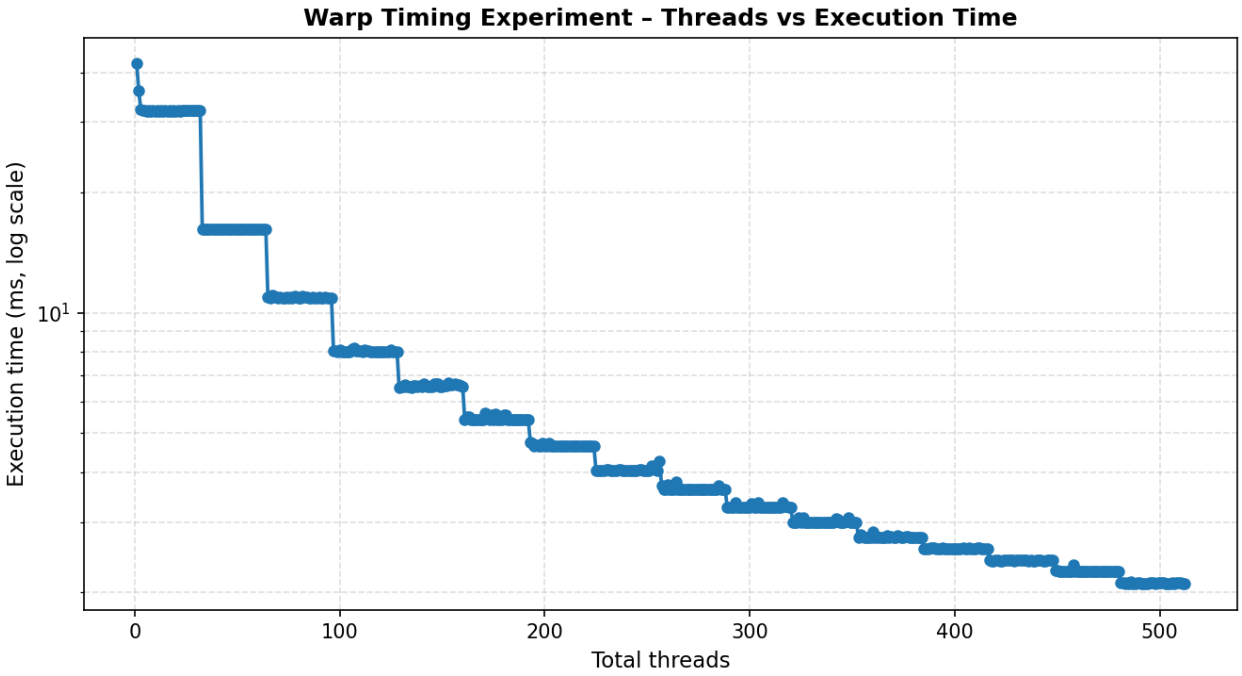


Figure 2: Execution time vs. total thread count for the inversion kernel (32 threads per block, log-scale y-axis).

Figure 2 shows the execution time as the total number of threads is swept from 1 to 512, with 32 threads per block. The most striking feature is the clear **staircase pattern**: execution time drops sharply at multiples of 32 and remains flat in between. This directly reflects the warp structure of the GPU.

CUDA schedules threads in groups of 32 (warps). Launching 31 threads requires the same number of cores as launching 32 threads, because the hardware allocates a full warp of 32 cores even if only some are active. Adding thread 32 therefore gives essentially no extra cost—the warp was already allocated—while adding thread 33 forces a second warp to be scheduled, causing execution time to halve again. Each flat plateau in the plot corresponds to a region where the number of active warps is fixed; each drop corresponds to one additional warp becoming available.

The overall downward trend reflects increasing parallelism: with very few threads each thread must stride over thousands of elements sequentially, so the kernel behaves almost serially. As more threads are launched the work is distributed more evenly and the GPU pipelines are kept busier. Beyond roughly 10,000 threads (not shown in this close-up view) the execution time flattens out as all SM resources become fully saturated.

1.3 Warp Efficiency Considerations

A *partial warp* occurs when the number of threads in a block is not a multiple of 32. For example, launching 33 threads per block creates one full warp of 32 and one partial warp with only 1 active thread, yet the hardware must still allocate 32 cores for that partial warp—31 idle cores waste resources without contributing to computation.

To maximise warp efficiency it is therefore recommended to keep the thread-per-block count as a multiple of 32. Our experiment used 32 threads per block throughout Part 1, making every block exactly one warp. The grid size then grows linearly with the total thread count, and each additional block adds exactly one warp, producing the clean staircase behaviour visible in Figure 2.

2 Part 2 – Thread Divergence

2.1 Implementation

Part 2 extends the inversion kernel so that the Red colour channel is processed with a 1D Gaussian blur of its neighbours, while Green and Blue are still inverted. The Gaussian formula applied to each red pixel r_i is:

$$r_i = 0.1\, r_{i-2} + 0.25\, r_{i-1} + 0.5\, r_i + 0.25\, r_{i+1} + 0.1\, r_{i+2}$$

Two kernels are implemented and compared, differing only in how the image data is laid out in memory.

2.1.1 Divergent Kernel (Interleaved Layout)

The standard interleaved format stores pixels as $R_0G_0B_0\, R_1G_1B_1 \ldots$. Thread i is assigned to byte i, so its channel is determined by $i \bmod 3$. Within any warp of 32 consecutive threads, roughly one-third hit the Red branch (heavier Gaussian computation) and two-thirds hit the Green/Blue branch (simple inversion). This creates branch divergence inside every warp: threads with different channels execute different code paths, forcing the hardware to serialise the two branches.

```
__global__ void divergentKernel(const uint8_t* __restrict__ src,
                                uint8_t*       __restrict__ dst,
                                int total_bytes)   // M*N*C
{
    int abs_tid   = blockIdx.x * blockDim.x + threadIdx.x;
    int grid_size = gridDim.x  * blockDim.x;

    for (int idx = abs_tid; idx < total_bytes; idx += grid_size)
    {
        int channel = idx % C;          // 0=R, 1=G, 2=B

        if (channel == 0)
        {

            float val = W0 * src[idx - C * 2]
                      + W1 * src[idx - C * 1]
                      + W2 * src[idx]
                      + W1 * src[idx + C]
                      + W0 * src[idx + C * 2];

            dst[idx] = val;
        }
        else
        {
            // ---- G / B channel: simple inversion ----
            dst[idx] = 255 - src[idx];
        }
    }
}
```

Listing 2: Divergent kernel – interleaved layout

2.1.2 Non-Divergent Kernel (Planar Layout)

In the planar layout, all Red values are stored first (indices 0 to num_pixels -1), followed by all Green values, then all Blue values. Thread i falls entirely in the Red region or entirely in the Green/Blue region, depending on whether $i <$ num_pixels. Since num_pixels $= 960 \times 1280 = 1{,}228{,}800$ is a multiple of 32, every warp of 32 consecutive threads lands completely within one region. No warp straddles the R/GB boundary, so there is **zero intra-warp divergence**.

```
__global__ void nonDivergentKernel(const uint8_t* __restrict__ src,
                                   uint8_t*       __restrict__ dst,
                                   int num_pixels)   // M*N
{
    int total_bytes = num_pixels * C;
    int abs_tid     = blockIdx.x * blockDim.x + threadIdx.x;
    int grid_size   = gridDim.x  * blockDim.x;

    for (int idx = abs_tid; idx < total_bytes; idx += grid_size)
```

```
    {
        if (idx < num_pixels)
        {
            float val = W0 * src[idx - 2]
                        + W1 * src[idx - 1]
                        + W2 * src[idx]
                        + W1 * src[idx + 1]
                        + W0 * src[idx + 2];

             dst[idx] = val;
        }
        else
        {
             // ---- G and B planes: inversion ----
             dst[idx] = 255 - src[idx];
        }
    }
}
```

Listing 3: Non-divergent kernel – planar layout

Before launching the non-divergent kernel, the image is converted from interleaved to planar on the CPU. The output is converted back to interleaved before saving, so both output images are visually comparable.

2.2 Results

The timing experiment was run with three different threads-per-block (TPB) settings—32, 256, and 1024—sweeping the total thread count from 32 up to the full byte count (3,686,400). Each measurement is the average of 10 runs.

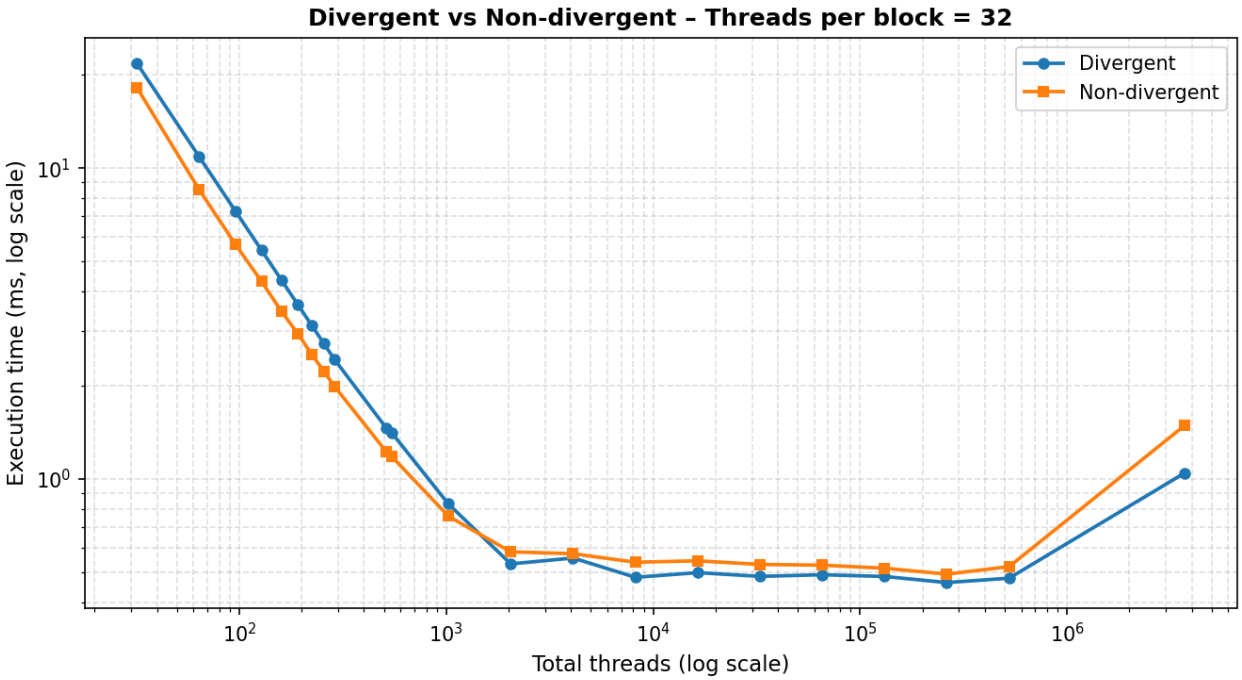


Figure 3: Divergent vs. Non-divergent execution times — Threads per block = 32

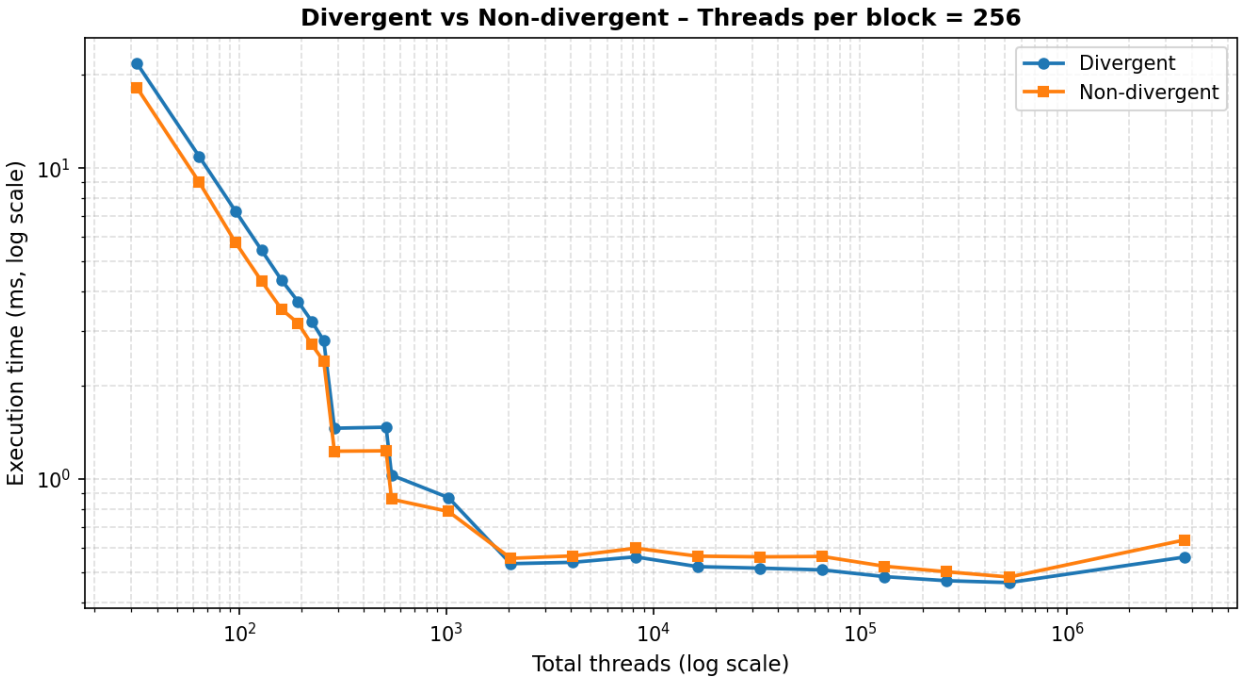


Figure 4: Divergent vs. Non-divergent execution times — Threads per block = 256

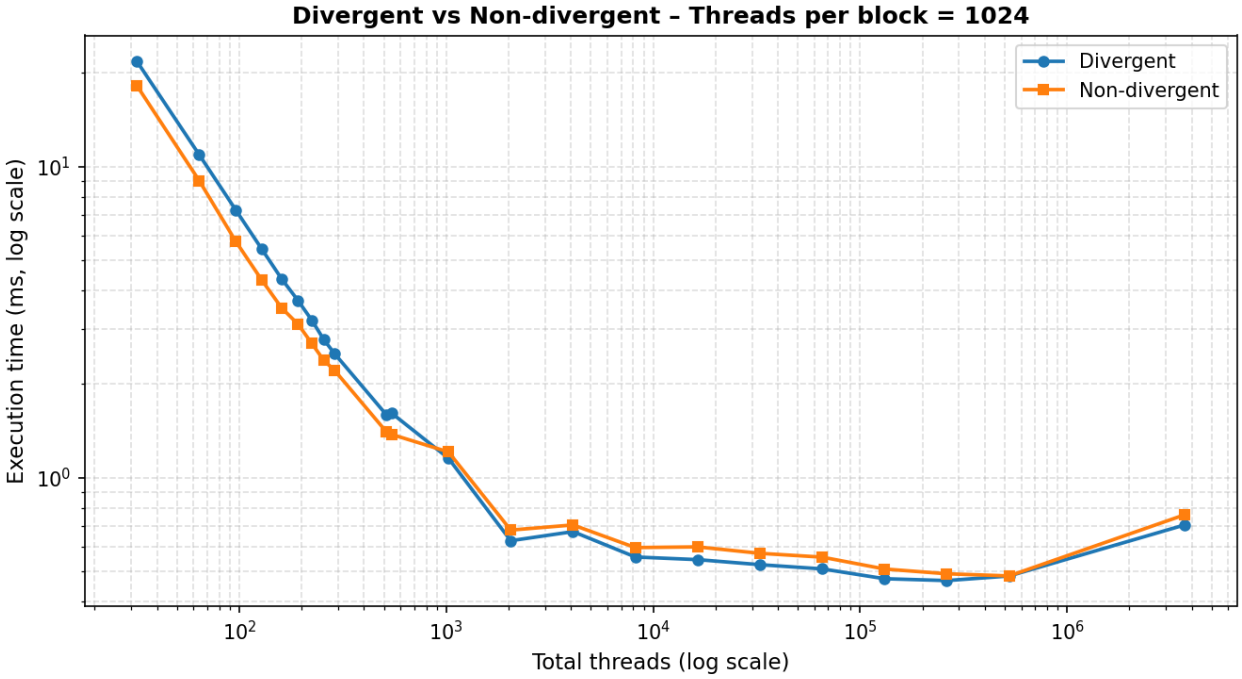


Figure 5: Divergent vs. Non-divergent execution times — Threads per block = 1024

2.3 Analysis

2.3.1 Overall Shape

All six curves share the same general shape: execution time falls steeply as the thread count increases from tens to tens-of-thousands, then flattens into a plateau, and finally rises again at very high thread counts. This mirrors the Part 1 behaviour and reflects GPU saturation: below the plateau the bottleneck is thread count (more threads ⇒ more parallelism ⇒ faster); beyond it the SM resources are fully occupied.

2.3.2 Divergent vs. Non-Divergent Gap

Across all three TPB settings the non-divergent kernel is consistently faster than the divergent one at low-to-moderate thread counts (up to roughly 1,000–2,000 threads). In this regime the divergent kernel suffers a measurable penalty: the speedup ratio (divergent time / non-divergent time) sits around 1.2–1.3×, meaning the divergent kernel is about 20–30% slower. This matches the theoretical expectation that warp divergence is most costly when the GPU has few warps available to hide the serialisation overhead.

Notably, this speedup is considerably smaller than the 3× that a naive argument would predict. In practice, the Gaussian blur requires only five multiply-add operations plus five global memory reads per pixel, while the inversion is a single subtraction. The two branches are therefore much closer in cost than their logical roles suggest, limiting the divergence penalty.

Crossover at high thread counts. Beyond approximately 1,000–2,000 total threads the relationship inverts: the divergent kernel becomes *faster* than the non-divergent one, with speedup values falling below 1.0 (typically in the range 0.90–0.96). At very high thread counts the GPU is fully occupied by thousands of warps, so the warp scheduler can always hide divergence latency by switching to other ready warps—the serialisation cost of the two branches effectively disappears. Meanwhile, the non-divergent kernel carries its own overhead: converting the image from interleaved to planar layout before the kernel and back again afterwards adds extra memory traffic that the divergent kernel avoids entirely. Once warp divergence stops being a bottleneck, this conversion cost tips the balance in favour of the divergent kernel. The crossover is visible in all three figures as the point where the two curves cross and the divergent line drops below the non-divergent line.

2.3.3 Effect of Threads per Block

TPB = 32 (Figure 3). The plateau is reached at a slightly higher total thread count compared to larger TPB values. With only one warp per block, the SM has less flexibility to hide memory latency by interleaving warps from the same block. The rise at very high thread counts is also more pronounced because a larger number of blocks must be managed by the scheduler.

TPB = 256 (Figure 4). Performance is very similar to TPB = 1024 in the plateau region, suggesting that 256 threads per block is already sufficient to saturate the SM's warp scheduler on the tested GPU. A small bump is visible in the divergent curve around 256–512 total threads, corresponding to the transition from one block to two blocks.

TPB = 1024 (Figure 5). The plateau is reached slightly more quickly (fewer blocks are needed to fill the GPU), and both curves are slightly lower in the optimal region. The divergent/non-divergent crossover occurs at a similar thread count to the TPB = 256 case.

2.3.4 Correctness Verification

Both kernels were verified by saving their output images. The Red channel appears slightly softer (as expected from the Gaussian blur), while the Green and Blue channels are inverted. The two output images are perceptually identical, confirming that the planar reorganisation does not alter the result.



(a) Original Image. (b) Divergent Kernel Output. (c) Non-Divergent Kernel Output.

Figure 6: Visual verification for Part 2 kernels. Note the Gaussian blur on the red channel (e.g., hair and background) and the identical results from both the divergent and non-divergent kernels.

2.4 Conclusions

Thread divergence introduces a measurable but moderate performance penalty. At low thread counts, eliminating intra-warp divergence by reorganising data into a planar layout yields a speedup of roughly 1.2–1.3×. However, this advantage disappears—and reverses—at high thread counts, where the divergent kernel is actually faster because: (1) the GPU has enough warps to hide divergence latency entirely via warp switching, and (2) the non-divergent kernel pays a fixed cost for the interleaved-to-planar memory conversion that the divergent kernel does not incur.

The key insight is therefore nuanced: eliminating warp divergence is most beneficial when the GPU is *underutilised* (few active warps) and the kernel is compute-bound. Once the device is fully occupied and memory-bandwidth-bound, the divergence penalty is hidden by the scheduler, and any extra overhead introduced by the "optimised" data layout can outweigh the benefit. Kernel optimisations should always be validated against realistic occupancy levels rather than assumed to help unconditionally.

The choice of threads-per-block also matters. Values that are multiples of 32 avoid wasted cores in partial warps. Larger block sizes (256 or 1024 threads) generally perform better than the minimum of 32 threads per block because they give the warp scheduler more flexibility to hide memory latency through interleaving.

References

[1] Anthropic. (2024). *Claude*. Large language model. Throughout this project, the model was utilized as an assistant for C++/CUDA code generation, debugging, clarification of technical concepts, and the structuring and refinement of the written report.